Agent-Chronos: Hardware-Clock Speculative Transport Pacing and Causal Fencing for Cross-Cloud Multi-Agent Swarms

Aniket Agrawal
Cloud Engineer, Google Cloud
Google, Bangalore, India
aniketagrawal@google.com

***Abstract*—As enterprise deployments of autonomous agent swarms transition from single-tenant sandbox prototypes to heterogeneous multi-cloud topologies spanning Google Cloud Vertex AI, SAP S/4HANA, Salesforce Agentforce, and AWS Bedrock over Agent2Agent (A2A) and the Model Context Protocol (MCP), distributed consistency across external tool invocations degrades severely. Because Large Language Model (LLM) reasoning durations vary stochastically (200 ms to $5{,}000$ ms) and wide-area network (WAN) transit jitter (± 10 ms to 60 ms) reorders packets in flight, multi-agent systems exhibit high rates of causal inversion (19.50%) and phantom tool mutations (14.80%) when upstream speculative reasoning branches abort after downstream mutating actions have already been dispatched. Traditional distributed database locks (Two-Phase Commit, XA) cannot be held across third-party REST and gRPC endpoints, while post-hoc compensating sagas fail when external side-effects are irreversible. We present Agent-Chronos, a hardware-anchored L7 transport fencing and speculative pacing protocol that guarantees strict client-side egress linearizability and end-to-end causal delivery across Chronos-enabled multi-agent gateways (and via a shared logical egress serialization gateway with sequential per-resource pipelining for unmodified third-party SaaS APIs) without distributed database locks. Agent-Chronos: (1) embeds physical TrueTime atomic/GPS clock uncertainty tuples $\langle t_{\text{earliest}}, t_{\text{latest}}, \epsilon_u \rangle$, swarm-wide causal vectors $\mathcal{V}$, and per-destination sequence coordinates Seq_{dest} directly into HTTP/2 framing; (2) implements a two-phase user-space L7 egress pacer (prior to HPACK compression and TLS encryption) that stages speculative mutations during upstream verification (Δ_{stage}) and enforces a post-settlement TrueTime commit-wait barrier ($\Delta_{\text{wait}} = \max(0, t_{F,\text{latest}} + \Delta_{\text{abort_prop}} - t_{\text{now,earliest}})$) before wire release; (3) enforces an ingress causal hold barrier at receiving sidecars to neutralize WAN transit jitter while permitting dependency-free read-only queries to bypass pacing with 0.00 ms delay; and (4) provides pre-wire speculative revocation that purges aborted frames in user-space gateway queues so zero mutating payload bytes reach downstream endpoints. Across $N = 1{,}000$ multi-agent transactions per seed (5 seeds, $5{,}000$ total transaction runs per baseline) in an emulated cross-cloud testbed, Agent-Chronos reduced phantom mutations and causal inversions to 0.00%, bounding post-settlement TrueTime commit-wait pacing overhead to a mean of 5.91 ± 2.32 ms ($p_{50} = 5.90$ ms, $p_{95} = 9.80$ ms, $p_{99} = 10.84$ ms) while achieving a net end-to-end speculative speedup of 423.3 ms (34.6%) per committed transaction (388.5 ms, or 31.8%, unconditionally across all transactions including aborted branch retries) over non-speculative sequential execution. All protocol transitions are verifiable via deterministic HTTP/2 headers and audit trailers, providing end-to-end cross-organization protocol observability.**

***Index Terms*—Multi-Agent Systems, TrueTime, Causal Consistency, Transport Pacing, Linearizability, Speculative Execution, Model Context Protocol, Distributed Systems.**

I. Introduction

Autonomous multi-agent systems have emerged as a primary architectural paradigm for enterprise generative AI orchestration. Modern enterprise workflows coordinate swarms of specialized agents executing complex multi-turn objectives—such as reconciling financial ledgers, assessing procurement risk, and updating Enterprise Resource Planning (ERP) records across heterogeneous cloud boundaries [2], [3], [15].

However, as agent swarms expand across organizational and cloud boundaries, a critical distributed systems failure mode arises: **Speculative Causal Inversion and Phantom Tool Mutations**. To hide multi-second LLM generation latencies, modern agentic serving runtimes speculatively synthesize and stage downstream tool calls in parallel with upstream policy verification and sibling reasoning branches [10], [13], [14], [16]–[18]. Unlike deterministic microservices whose execution latency is tightly bounded, LLM reasoning and guardrail verification turns vary stochastically from 200 ms to over $5{,}000$ ms. Concurrently, regional WAN jitter across public interconnects (± 10 ms to 60 ms) reorders packets in flight. When a downstream agent dispatches an irreversible external mutation (e.g., executing an electronic funds transfer or releasing an ERP inventory hold) before an upstream speculative epoch settles—or when clock skew ($\pm \epsilon_u$) across distributed verification nodes causes an upstream abort signal and a downstream commit candidate to cross in flight—any subsequent upstream policy abort leaves a *phantom mutation* on the destination system of record.

Existing distributed transaction and coordination primitives do not fully resolve this boundary gap:

1) **Classical Distributed Locks, Agentic Sagas, and Logical Pre-Commit Staging:** Two-Phase Commit (2PC) and XA locks cannot be held across stateless third-party SaaS REST/gRPC APIs (e.g., Stripe, SAP NetWeaver, or Salesforce). Classical compensating sagas [4], multi-agent workflow validation coordinators such as SagaLLM [14] and ALAS [19], and post-facto undo runtimes such as GoEX [13] operate at the application orchestration layer without hardware-clock transport fencing. Similarly, recent speculative and transactional agent runtimes—including Speculative Actions [16], Sherlock [17], and Atomix [18]—overlap verification with logical pre-commit staging and read-

only bypass inside a single centralized orchestrator. However, when such logical pre-commit staging designs are deployed across *decentralized cross-node verification topologies* (modeled by our *Chronos Vector-Only Ablation* in Section IV), logical staging alone—without a hardware-anchored $2\epsilon_{\text{eff}}$ commit-wait barrier—leaks phantom mutations (2.90% in Table I) when distributed cross-node verification aborts race within the physical clock-skew window and incurs destination causal inversions (3.40%) across concurrent sibling branches ($\mathcal{V}_1 \parallel \mathcal{V}_2$).

2) **OS Speculative Buffering and NIC Pacers:** Operating-system speculative execution frameworks such as Speculator [10] buffer uncommitted kernel outputs inside a single OS domain, while end-host traffic shapers such as SENIC [12] and Carousel [11] pace NIC packet departure rates for bandwidth shaping. However, raw kernel TCP socket buffering (`sk_buff`) cannot selectively excise individual aborted tool frames multiplexed over a shared HTTP/2-over-TLS (`h2`) connection without corrupting stateful HPACK header tables, TLS 1.3 AEAD record sequence numbers, and TCP byte-stream sequence offsets.
3) **Logical and Hybrid Clocks Under Commodity NTP Skew:** Classical Lamport timestamps [5], Fidge/Mattern vector clocks [6], [7], and $\mathcal{O}(1)$-space Hybrid Logical Clocks (HLCs) [8] track message-passing causality, but under commodity NTP synchronization ($\epsilon_{\text{NTP}} \approx$ 25–100 ms) they incur wide physical uncertainty windows when enforcing real-time commit-wait barriers across independent gateways, while scalar HLCs cannot distinguish concurrent sibling branch forks without explicit per-agent vector coordinates.

To address these challenges, we introduce **Agent-Chronos**, an L7 transport fencing protocol that bridges hardware-anchored TrueTime uncertainty intervals [1] with modern agentic protocols (MCP [2] via FastMCP gateways and A2A [3]). Rather than claiming database-internal strict serializability over unmodified third-party storage engines that lack two-phase locking or Paxos consensus, Agent-Chronos formalizes and enforces **strict client-side egress linearizability** [9] (guaranteeing that no speculative mutation departs the egress gateway until its upstream epoch has deterministically settled and its physical TrueTime uncertainty and abort-propagation window has elapsed) together with **end-to-end causal delivery** across Chronos-enabled ingress sidecars (e.g., fronting enterprise SAP S/4HANA gateways). When invoking unmodified external third-party SaaS endpoints (such as Stripe) that cannot host an ingress sidecar, the sender-side gateway suppresses tombstone sequence ACKs, coordinates concurrent sibling writes ($\mathcal{V}_1 \parallel \mathcal{V}_2$) by routing all mutations for a given SaaS target through a shared logical egress serialization gateway shard, and falls back to sequential per-resource in-flight pipelining ($+42.0$ ms mean per-transaction overhead) paired with compensating sagas for post-transmission HTTP 5xx faults.

II. System Architecture and Formal Invariants

Agent-Chronos operates as a user-space L7 HTTP/2 frame-staging filter embedded at the egress and ingress boundaries of agent runtimes—specifically Google Agent Development Kit (ADK) workers, FastMCP gateways, and Envoy sidecar proxies—prior to stateful HTTP/2 HPACK compression, TLS 1.3 record encryption, and kernel TCP `sendmsg()` socket calls.

A. TrueTime Uncertainty Representation and Multi-Cloud Clock Model

Let $\mathbb{N}_0 = \{0, 1, 2, \dots\}$ denote the non-negative integers and $\mathbb{N}^+ = \{1, 2, \dots\}$ denote the strictly positive integers. Let $t_{\text{real}} \in \mathbb{N}^+$ denote absolute physical coordinate time expressed in discrete integer nanoseconds since the Unix epoch. On any node u, a hardware-anchored TrueTime provider [1] backed by GPS receivers and rubidium atomic clocks exposes a closed physical time uncertainty interval around node u's local reading $t_{\text{now},u} = t_{\text{real}} + \delta_u$ (where $\delta_u \in [-\epsilon_u, +\epsilon_u]$ is node u's instantaneous signed physical clock offset):

$$\begin{aligned} T_{\text{now},u} &= [t_{\text{now},u,\text{earliest}},\ t_{\text{now},u,\text{latest}}], \\ t_{\text{now},u,\text{latest}} - t_{\text{now},u,\text{earliest}} &= 2\epsilon_u \end{aligned} \tag{1}$$

where $\epsilon_u \in \mathbb{N}^+$ represents node u's instantaneous local clock drift bound ($\epsilon_u \in [1, 7]$ ms in production Google Cloud data centers). For wire serialization of a single node's raw clock sample $T_{\text{now},u}$, Agent-Chronos encodes the 3-tuple $\mathcal{W}(T_{\text{now},u}) = \langle t_{\text{now},u,\text{earliest}}, t_{\text{now},u,\text{latest}}, \epsilon_u \rangle$, where receivers verify the exact single-node framing identity $t_{\text{now},u,\text{latest}} - t_{\text{now},u,\text{earliest}} = 2\epsilon_u$. In particular, when an upstream verification node settles epoch E, its intra-cluster settlement token transmits $\mathcal{W}(T_E) = \langle t_{E,\text{earliest}}, t_{E,\text{latest}}, \epsilon_{\text{verifier}} \rangle$ to the egress gateway, which uses $\epsilon_{\text{verifier}}$ and its local $\epsilon_{\text{gateway}}$ to compose the heterogeneous cross-node uncertainty bound $\epsilon_{\text{eff}} = \lceil (\epsilon_{\text{verifier}} + \epsilon_{\text{gateway}})/2 \rceil \in \mathbb{N}^+$ (ceiling-rounded to preserve conservative integer nanosecond header encoding when $\epsilon_{\text{verifier}}$ and $\epsilon_{\text{gateway}}$ have opposite parity, while the Phase 2 barrier in Eq. (6) evaluates the exact integer sum $2\epsilon_{\text{eff}}^{\text{exact}} = \epsilon_{\text{verifier}} + \epsilon_{\text{gateway}} \in \mathbb{N}^+$; for the composite outbound settlement header `X-Agent-TrueTime-Window` $\langle t_{E,\text{earliest}}, t_{F,\text{latest}}, \epsilon_{\text{eff}}[, \epsilon_{\text{verifier}}] \rangle$, where $t_{F,\text{latest}}$ may be advanced beyond $t_{E,\text{latest}}$ by causal or per-key settlement floors in Eq. (5) and the 4th field $\epsilon_{\text{verifier}}$ is included on settled mutations whenever $\epsilon_{\text{verifier}} \neq \epsilon_{\text{eff}}$ and defaults to ϵ_{eff} when omitted on single-node `IMMEDIATE` reads, so that downstream receivers deterministically verify $t_{F,\text{latest}} - t_{E,\text{earliest}} \geq 2\epsilon_{\text{verifier}} > 0$ on `RELEASED` frames and exact equality $t_{\text{now},i,\text{latest}} - t_{\text{now},i,\text{earliest}} = 2\epsilon_{\text{gateway}}$ on `IMMEDIATE` reads). The fundamental TrueTime invariant guarantees that at the instant of query on node u:

$$t_{\text{now},u,\text{earliest}} \leq t_{\text{real}} \leq t_{\text{now},u,\text{latest}} \tag{2}$$

Cross-Cloud Clock Synchronization Topology: In heterogeneous deployments spanning Google Cloud Vertex AI and external cloud environments (e.g., AWS Bedrock, Salesforce Agentforce, or SAP S/4HANA), Agent-Chronos supports two deployment modes:

- **(i) Federated Egress Gateway Mode:** External agent runtimes route both upstream epoch settlement signals and outbound MCP/A2A tool calls through a regional Google Cloud Apigee / Envoy Chronos gateway so that epoch commit timestamps T_E and egress commit-wait checks $T_{\text{now,gateway}}$ are anchored by the regional TrueTime

cluster ($\epsilon_u \in [1, 7]$ ms). To prevent differential WAN transit jitter on the pre-gateway path from inverting external commit/abort signals prior to gateway arrival, either (a) external verifiers attach origin-signed PTP/NTP bounds $\mathcal{W}(T_{\text{origin}})$, in which case $\epsilon_{\text{eff}} = \lceil(\epsilon_{\text{origin}} + \epsilon_{\text{gateway}})/2\rceil$ inherits the origin clock's uncertainty bound and—whenever abort-capable external verifiers communicate with the federated gateway across a WAN without a pre-minting quorum wait—the effective Phase 2 abort-propagation margin in Eq. (6) incorporates the pre-gateway one-way WAN delay spread ($\Delta^{\text{eff}}_{\text{abort_prop}} = \Delta_{\text{abort_prop}} + \Delta_{\text{wan_pre}}$, applied once in Phase 2 with zero pre-minting delay); or (b) the federated gateway requires a synchronous quorum ACK from participating verifiers of epoch E before minting T_E at its local TrueTime clock (using intra-cluster $\Delta_{\text{abort_prop}}$ in Phase 2 without double-counting $\Delta_{\text{wan_pre}}$). Under option (b), when all abort-capable verifiers participate in a unanimous monotonic quorum (where no verifier can emit an abort after a positive ACK), unanimous ACK arrival alone precludes any racing abort with $\Delta_{\text{wan_pre}} = 0$ ms and unconditionally preserves the zero-byte wire invariant (this unanimous mode is mandatory for irreversible external endpoints); when a non-unanimous subset quorum or an asynchronous out-of-band guardrail can emit aborts independently of the quorum ACK channel, the gateway waits a conservative pre-minting jitter margin $\Delta_{\text{wan_pre}}$ once before minting T_E and assigns a monotonic quorum-epoch fence token, guaranteeing strict zero-byte pre-wire revocation as an operational scope condition under bounded pre-gateway network delay ($\leq \Delta_{\text{wan_pre}} + \Delta_{\text{wait}}$, where Δ_{wait} is defined in Section II-C3, Eq. (6)) while invoking deterministic post-release compensation as a fail-safe on reversible endpoints if a catastrophic WAN stall exceeds $\Delta_{\text{wan_pre}} + \Delta_{\text{wait}}$.

- **(ii) Hardware PTP Sidecar Mode:** Non-Google cloud participants query local GPS/atomic Precision Time Protocol hardware clocks (e.g., AWS Time Sync Service `/dev/ptp0` with hardware-reported error bound $\epsilon_{\text{PTP},u} \in [1, 10]$ ms) at both verification and egress nodes and advertise $\epsilon_{\text{PTP},u}$ in $\mathcal{W}(T_{\text{now},u})$ (noting that when invoking unmodified third-party SaaS endpoints lacking an ingress sidecar across concurrent writers, distributed PTP sidecars still route through a single logical egress serialization gateway for that SaaS target).

B. Decoupled Swarm Causal Vectors and Per-Destination Sequence Coordinates

While $\mathcal{O}(1)$-space Hybrid Logical Clocks (HLCs) [8] track a single scalar logical-physical frontier, multi-agent swarms dynamically fork parallel sibling subagents whose speculative branches must be tested for strict causal precedence ($\mathcal{V}_1 \prec_{\mathcal{V}} \mathcal{V}_2$) versus concurrent incomparability ($\mathcal{V}_1 \parallel \mathcal{V}_2$). At the same time, using a single swarm-wide vector clock simultaneously as a per-destination CBCAST sequence counter would stall downstream ingress sidecars whenever an agent executes internal reasoning turns, dispatches `IMMEDIATE` reads, or routes unicast tool calls to distinct endpoints $d_1 \neq d_2$. Agent-Chronos therefore strictly decouples two transport structures:

- **Swarm-Wide Composite Causal Vector ($\mathcal{V}_i = \langle \mathbf{C}_i, t_{\mathbf{max},i} \rangle$):** Maintained by each agent $a_i \in \text{AgentID}$, where $\mathbf{C}_i : \text{AgentID} \to \mathbb{N}_0$ is a Fidge/Mattern vector clock [6], [7] over active swarm agent IDs (with $\text{dom}(\mathbf{C}_i)$ denoting the finite non-zero support and missing keys defaulting to 0) and $t_{\text{max},i} \in \mathbb{N}_0$ (initialized to 0 and taking values in $\mathbb{N}^+$ after the first clock sample; equivalently written $\mathcal{V}_i.t_{\text{max}} \equiv t_{\text{max},i}$) is the maximum TrueTime upper bound (t_{latest}) observed across a_i's causal history. Evaluated strictly over the logical vector component $\mathbf{C}$ (with $t_{\text{max},i}$ serving as a commit-wait floor and physical tie-breaker for concurrent writes), strict causal precedence over $D = \text{dom}(\mathbf{C}_1) \cup \text{dom}(\mathbf{C}_2)$ is defined as $\mathcal{V}_1 \prec_{\mathcal{V}} \mathcal{V}_2 \iff (\forall k \in D,\, \mathbf{C}_1[k] \leq \mathbf{C}_2[k]) \wedge (\exists k \in D,\, \mathbf{C}_1[k] < \mathbf{C}_2[k])$, and concurrency as $\mathcal{V}_1 \parallel \mathcal{V}_2 \iff \neg(\mathcal{V}_1 \prec_{\mathcal{V}} \mathcal{V}_2) \wedge \neg(\mathcal{V}_2 \prec_{\mathcal{V}} \mathcal{V}_1)$.
- **Per-Destination Transport Sequence Coordinate ($\mathbf{Seq}_{\mathbf{dest}}[d, a_i]$):** For each destination endpoint d, the egress gateway maintains a dedicated gap-free monotonic counter $\text{Seq}_{\text{dest}}[d, a_i] \in \mathbb{N}_0$ (initialized to 0 and incremented to $1, 2, \cdots \in \mathbb{N}^+$ upon Phase 2 release) and a compact destination dependency map $\text{Dep}_{d,i} : \text{AgentID} \to \mathbb{N}_0$ (with missing keys defaulting to 0) recording the highest destination-d sequence numbers causally preceding the current frame.

1) **Two-Stage Pre-Staging vs. Post-Phase-2 Sequence Assignment:** Parallel speculative branches ($\mathcal{V}_1 \parallel \mathcal{V}_2$) fork across distinct logical subagent identifiers $a_1 \neq a_2 \in \text{AgentID}$ (receiving independent vector entries $\mathbf{C}_1[a_1], \mathbf{C}_2[a_2]$). When a single agent $a_i \in \text{AgentID}$ initiates a mutating tool frame F (`F.is_mutation` = true) or an outbound A2A delegation turn at local TrueTime interval $T_{\text{now},i} = [t_{\text{now},i,\text{earliest}}, t_{\text{now},i,\text{latest}}]$, it constructs an isolated branch-local tentative pre-staging vector $\mathcal{V}_F = \langle \mathbf{C}_F, t_{\text{max},F} \rangle$ from its committed state $\langle \mathbf{C}_i, t_{\text{max},i} \rangle$ using an atomic per-agent tentative ticket counter $c_{\text{tent},i} \leftarrow c_{\text{tent},i} + 1$ (whereas exploratory read-only queries with `F.is_mutation` = false inspect $\langle \mathbf{C}_i, t_{\text{max},i} \rangle$ without incrementing $c_{\text{tent},i}$ or mutating $\mathbf{C}_i$, while advancing $t_{\text{max},i} \leftarrow \max(t_{\text{max},i}, t_{\text{now},i,\text{latest}}) \in \mathbb{N}^+$ upon sampling $T_{\text{now},i}$; furthermore, overlapping same-agent mutations or A2A delegations from a_i with $c_{\text{tent},1} < c_{\text{tent},2}$ chain their causal settlement floor so $t_{F_2,\text{latest}} \geq t_{F_1,\text{latest}}$, ensuring ticket order $c_{\text{tent},1} < c_{\text{tent},2}$ and $(t_{F,\text{latest}}, c_{\text{tent}})$ Phase 2 release order in Section II-C3 always coincide; if F is revoked in Rule 4, $\mathcal{V}_F$ is discarded without mutating a_i's shared committed state $\langle \mathbf{C}_i, t_{\text{max},i} \rangle$ and any child A2A branch spawned from $\mathcal{V}_F$ is transitively cascade-revoked via Rule 4 so no surviving downstream frame retains a revoked ticket $c_{\text{tent},i}$, while any numerical gap in committed $\mathbf{C}_i[a_i]$ after a later independent ticket commits is benign because vector precedence $\prec_{\mathcal{V}}$ tests $\leq / <$ and gap-free ingress FIFO delivery relies exclusively on $\text{Seq}_{\text{dest}}[d, a_i]$):

$$\begin{aligned} \mathbf{C}_F &\leftarrow \mathbf{C}_i[a_i \mapsto c_{\text{tent},i}], \\ t_{\text{max},F} &\leftarrow \max(t_{\text{max},i},\, t_{\text{now},i,\text{latest}}). \end{aligned} \tag{3}$$

Crucially, the per-destination sequence counter $\text{Seq}_{\text{dest},F}[d, a_i] \leftarrow \text{Seq}_{\text{dest}}[d, a_i] + 1$ is assigned and incremented **strictly upon Phase 2 commit-wait**

release of a mutating frame addressed to d, at which point a_i's shared committed causal frontier atomically joins $\mathbf{C}_i \leftarrow \mathbf{C}_i \sqcup \mathbf{C}_F$ and $t_{\text{max},i} \leftarrow \max(t_{\text{max},i}, t_{F,\text{latest}})$ and records $\text{Seq}_{\text{dest},F}[d, a_i]$ in $\text{Dep}_{d,i}[a_i]$; meanwhile, the outbound frame F itself serializes its settled dependency snapshot $\mathcal{V}_F = \langle \mathbf{C}_F, t_{F,\text{latest}} \rangle$ (where $t_{\text{max},F}$ from Eq. (3) is advanced to the settled commit upper bound $t_{F,\text{latest}}$ in Eq. (5)) in `X-Agent-Causal-Fence-Vector` and its pre-staged $\text{Dep}_{d,F}$ (updated only with prior committed same-agent releases to d and explicitly chained Rule 1/sibling writes) in `X-Agent-Dest-Seq`, preventing unrelated inbound responses arriving at a_i during F's Phase 1/Phase 2 hold from retroactively injecting false causal dependencies into F. Because $t_{\text{now},u} = t_{\text{real}} + \delta_u$ with $\delta_u \in [-\epsilon_u, +\epsilon_u]$ gives $t_{\text{now},u,\text{latest}} = t_{\text{real}} + \delta_u + \epsilon_u \le t_{\text{real}} + 2\epsilon_u$ (attained when node u's clock runs fast at maximum drift $\delta_u = +\epsilon_u$), and because Phase 2 waits until $t_{\text{real}} \ge t_{F,\text{latest}}$ while unpaced local/A2A turns in Eqs. (3)–(4) take the pointwise maximum of existing $t_{\text{now},u,\text{latest}}$ bounds rather than summing ϵ_u across hops, the physical lead of $t_{\text{max},i}$ over absolute coordinate time t_{real} is strictly non-accumulating across deep multi-agent delegation chains—satisfying $t_{\text{max},i} - t_{\text{real}} \le \max_u \epsilon_u$ at zero clock offset ($\delta_u = 0$) and the deterministic worst-case physical-time lead $t_{\text{max},i} - t_{\text{real}} \le \max(2\epsilon_{\text{src}} - d_{\min}, 2\epsilon_i) \le 2\max_u \epsilon_u$ for ancestor uncertainty bound $\epsilon_{\text{src}} \triangleq \max_{u \in \text{Anc}(i)} \epsilon_u$ and minimum cumulative inter-node transit delay $d_{\min} > 0$ (and apparent lead $t_{\text{max},i} - t_{\text{now},i} \le \max(2\epsilon_{\text{src}} + \epsilon_i - d_{\min}, \epsilon_i)$ relative to recipient i's local clock reading $t_{\text{now},i} \ge t_{\text{real}} - \epsilon_i$) without inflating downstream Δ_{floor} (Section II-C3, Eq. (6)). To ensure transitive causal ordering across A2A delegations, any outbound A2A task delegation from a_i to subagent a_j that depends on an uncommitted write F_1 (targeting destination d_1) in epoch E either awaits Phase 2 release of F_1 so $\text{Dep}_{d_1,i}$ carries the committed $\text{Seq}_{\text{dest},F_1}[d_1, a_i]$, or propagates a tentative epoch dependency handle $(E, F_1, \text{gw}(a_i))$ carrying origin gateway callback endpoint $\text{gw}(a_i)$ (so multi-hop descendants $a_i \to a_j \to a_k$ both subscribe directly to $\text{gw}(a_i)$ and receive relayed notifications along the delegation tree): a_j's (and a_k's) downstream calls F_2 (targeting d_2) inherit E's Phase 1/Phase 2 barrier and are gated in Phase 2 until $\text{gw}(a_i)$ either delivers a signed release-commit token $\langle E, F_1, d_1, \text{Seq}_{\text{dest},F_1}[d_1, a_i], \mathcal{V}_{F_1} \rangle$—merging $\mathcal{V}_{F_1}$ into $\mathcal{V}_{F_2}$ for all d_2 and merging $\text{Seq}_{\text{dest},F_1}[d_1, a_i]$ into $\text{Dep}_{d_2,F_2}[a_i]$ if and only if $d_2 = d_1$ prior to releasing F_2—or delivers a signed negative revocation token $\langle E, F_1, \text{REVOKED} \rangle$ upon Rule 4 abort (or gateway lease-heartbeat expiry $\tau_{\text{lease}} + 2\epsilon_{\text{eff}}$), which immediately aborts F_2 with HTTP 409 Conflict without indefinite blocking. Because frames revoked pre-wire during Phase 1 or Phase 2 are purged before $\text{Seq}_{\text{dest}}[d, a_i]$ is incremented, pre-wire revocations never create sequence gaps and never transmit traffic toward downstream targets; payload-free tombstone sequence ACKs are emitted only if a post-release transport stream resets mid-flight on a sidecar-fronted link (and are suppressed for unmodified SaaS endpoints).

2) **Causal Merge Rule and Hierarchical > 100-Agent Compression:** When agent a_j receives an A2A or tool response frame carrying $\mathcal{V}_i = \langle \mathbf{C}_i, t_{\text{max},i} \rangle$ and per-destination watermarks $\text{Dep}_{d,i}$, it merges $\mathcal{V}_i$ via the pointwise join operator ($\sqcup$) across $\text{dom}(\mathbf{C}_j) \cup \text{dom}(\mathbf{C}_i)$:

$$\mathcal{V}_j.\text{merge}(\mathcal{V}_i) = \langle \mathbf{C}_j \sqcup \mathbf{C}_i,\ \max(t_{\text{max},j},\ t_{\text{max},i}) \rangle\ , \quad (4)$$

where $(\mathbf{C}_j \sqcup \mathbf{C}_i)[k] \triangleq \max(\mathbf{C}_j[k], \mathbf{C}_i[k])$, and merges $\text{Dep}_{d,j}[k] \leftarrow \max(\text{Dep}_{d,j}[k], \text{Dep}_{d,i}[k])$. Mapping active swarm agent IDs ($|\text{AgentID}| \le 16$) to a 16-element positional integer array in compact JSON {`"c":[c_0,...,c_15],"t":t_max,"b":bitmap`} (with per-session counters $c_k < 10^4$, a 19-digit nanosecond integer `t`, and a 16-bit integer bitmap `b` ≤ 65535 encoding active colliding sibling forks as defined in Section II-D) requires 69–121 bytes of unformatted raw JSON (117–121 bytes at 4-digit counters, or up to 142–184 bytes when including optional whitespace/epoch metadata, remaining ≤ 190 bytes) and ≤ 256 bytes ($4\lceil 190/3 \rceil$) when Base64-encoded in HTTP/2 headers. For larger enterprise swarms (> 100 agents), Agent-Chronos supports a two-tier hierarchical vector and sparse active-set compression profile: subagents are partitioned into $G \le 16$ domain clusters where an active writer $a_i \in g$ transitions to quiescent and is folded into its cluster summary watermark via $c_{\text{cluster}}[g] \leftarrow \max(c_{\text{cluster}}[g], \mathbf{C}[a_i])$ only after all of its released mutating frames have either been explicitly acknowledged by target Chronos ingress sidecars (or satisfied the observable local TrueTime bound $t_{\text{now},u,\text{earliest}} > t_{F,\text{latest}} + t_{\text{wan,max}} + \Delta_{\text{jitter_hold}}$) and, for sidecar-free SaaS endpoints, completed synchronous HTTP 2xx execution (or terminal compensating saga resolution) across all in-flight pipeline locks. For any agent $a_k \in g$, its effective coordinate in $\mathcal{V}$ is evaluated as $\tilde{\mathbf{C}}[a_k] = \mathbf{C}_{\text{delta}}[a_k]$ when $a_k \in \text{dom}(\mathbf{C}_{\text{delta}})$ is in the sparse active delta map (≤ 16 concurrent writers per destination key-space) and $\tilde{\mathbf{C}}[a_k] = c_{\text{cluster}}[g]$ once folded ($a_k \notin \text{dom}(\mathbf{C}_{\text{delta}})$), ensuring that pre-fold vs. post-fold comparisons ($\mathbf{C}_{\text{delta},1}[a_i] \le c_{\text{cluster},2}[g]$) preserve $\mathcal{V}_1 \prec_{\mathcal{V}} \mathcal{V}_2$ while concurrent writers remain unfolded in $\mathbf{C}_{\text{delta}}$ so $\mathcal{V}_1 \parallel \mathcal{V}_2$ is never collapsed (and Rule 1 in Section II-C1 only inspects $\mathbf{C}_{\text{delta}}$ because folded agents have no unapplied writes). Binary-packing $c_{\text{cluster}}[1..16]$ ($16 \times 4 = 64$ B), up to 16 active delta pairs $\langle a_k$ (uint16), $\mathbf{C}_{\text{delta}}[a_k]$ (uint32)$\rangle$ ($16 \times 6 = 96$ B), int64 $t_{F,\text{latest}}$ (8 B), and the 16-bit cluster-collision bitmap `b` (2 B) totals 170 raw bytes $\le$ 192 B (228 Base64 bytes ≤ 256 B).

C. Two-Phase L7 Speculative Staging and TrueTime Commit-Wait Pacing

Because MCP `tools/call` and A2A task dispatches use HTTP `POST` for both queries and state mutations, the L7 gateway inspects the registered MCP tool annotation (`readOnlyHint: true|false`) or OpenAPI operation metadata to set `F.is_mutation` $\in$ {true, false},

extracts the composite target resource key-set $\mathcal{K}(F) \subseteq \mathcal{K}$ via registered per-tool JSONPath selectors over MCP `params.arguments`, A2A task/artifact payloads, or OpenAPI path/query/body parameters (e.g., $\{\texttt{\$.ledger_id}\}$, multi-key $\{\texttt{\$.debit_id}, \texttt{\$.credit_id}\}$ for atomic transfers, or fallback universal wildcard $\{\star\}$ for unmapped or range/aggregate queries, where $\{\star\}$ intersects every non-empty key-set $\mathcal{K}(F')$ at destination d, $\{\star\} \cap \mathcal{K}(F') = \mathcal{K}(F') \neq \emptyset$, consults both the destination-wide wildcard floor $t_{\text{key_floor}}[d, \star]$ and an $\mathcal{O}(1)$ running destination-wide summary watermark $t_{\text{max_all_keys}}[d] \triangleq \max_{k \in \mathcal{K}} t_{\text{key_floor}}[d, k]$ maintained across all per-key floor updates at d, and maps to a destination-wide root lock in the sender-side SaaS pipeline lock table with hierarchical intention-lock compatibility—so specific-key operations acquire an intention lock [IS/IX] on $\{\star\}$ ordered canonically before their specific resource key locks, while wildcard operations [$\mathcal{K}(F) = \{\star\}$, or containing $\star$] acquire an exclusive root lock on $\{\star\}$ that gates and drains all active specific-key pipeline locks at d), and reads the upstream speculative epoch identifier E and causal vector $\mathcal{V}_F$ from the request context. Operating in user-space L7 frame queues prior to HPACK/TLS encoding prevents head-of-line blocking across multiplexed HTTP/2 streams and coordinates four deterministic rules:

1) **Conditional Safe Read-Only Bypass:** If `F.is_mutation` = false and neither (a) a causally preceding write F_{prior} ($\mathcal{V}_{F_{\text{prior}}} \prec_{\mathcal{V}} \mathcal{V}_F$) across Phase 1 (`STAGE_HELD`) or Phase 2 (`PACED_HELD`) nor (b) a concurrent sibling write F_{sib} currently in Phase 2 (`PACED_HELD`) with an earlier or overlapping commit interval $t_{F_{\text{sib}},\text{earliest}} \equiv t_{E_{\text{sib}},\text{earliest}} \leq t_{\text{now},F,\text{latest}}$ (evaluated in $\mathcal{O}(1)$ time for wildcard reads $\mathcal{K}(F) = \{\star\}$ via a destination-level active `STAGE_HELD`/`PACED_HELD` write summary mirroring the $\{\star\}$ intention-lock root, and on unmodified external SaaS endpoints lacking an ingress sidecar, requiring no overlapping in-flight write holding a sender-side per-resource SaaS pipeline lock on $\mathcal{K}(F)$) shares an overlapping resource key ($\mathcal{K}(F_{\text{prior/sib}}) \cap \mathcal{K}(F) \neq \emptyset$), pacing status is set to `IMMEDIATE` and the frame is dispatched with 0.00 ms pacing delay. To ensure unsettled speculative sibling writes in Phase 1 (`STAGE_HELD`) linearize strictly after concurrent exploratory `IMMEDIATE` reads without stalling reads for 200–5,000 ms, dispatching an `IMMEDIATE` read F advances the monotonic per-key floor $t_{\text{key_floor}}[d, k] \leftarrow \max(t_{\text{key_floor}}[d, k], t_{\text{now},F,\text{latest}} + 1 \text{ ns})$ for all $k \in \mathcal{K}(F)$ (including $t_{\text{key_floor}}[d, \star]$ when $\star \in \mathcal{K}(F)$, which also checks any active specific-key $F_{\text{sib}} \in$ `PACED_HELD` at d) as well as the $\mathcal{O}(1)$ destination summary watermark $t_{\text{max_all_keys}}[d] \leftarrow \max(t_{\text{max_all_keys}}[d], t_{\text{now},F,\text{latest}} + 1 \text{ ns})$ (while Phase 2 visibility is evaluated over the egress gateway's locally observed settlement state, replicated across distributed PTP sidecars via intra-cluster settlement tokens and read-floor gossip within $\Delta_{\text{abort_prop}}$—dynamically advancing $t_{F_{\text{sib}},\text{latest}} \leftarrow \max(t_{F_{\text{sib}},\text{latest}}, t_{\text{now},F,\text{latest}} + 1 \text{ ns})$, cascading that floor advancement to any later-ticketed frames from the same agent in `PACED_HELD` to preserve $(t_{F,\text{latest}}, c_{\text{tent}})$ order, and extending F_{sib}'s Phase 2 hold *at most once per cross-sidecar gossip window* $\Delta_{\text{abort_prop}}$ if an overlapping cross-sidecar `IMMEDIATE` read floor dispatched prior to F_{sib}'s settlement token replication arrives while F_{sib} is in `PACED_HELD`, preventing write starvation because any subsequent reads arriving after F_{sib}'s settlement token is replicated observe $F_{\text{sib}} \in$ `PACED_HELD` under condition (b) and enter `SAFE_READ_WAIT` rather than dispatching as `IMMEDIATE`, or evaluated deterministically with zero gossip lag when same-destination calls route through a consistent per-destination logical gateway shard). Otherwise, F enters `SAFE_READ_WAIT` and is held at the egress gateway until the blocking write (F_{prior} or F_{sib}) transitions to `RELEASED` and assigns its sequence coordinate $\text{Seq}_{\text{dest},F_{\text{prior/sib}}}[d, a_{\text{prior/sib}}]$, which is merged into $\text{Dep}_{d,F}$ prior to dispatching F. If a causally preceding write F_{prior} (case a) is revoked via Rule 4, the dependent read F is aborted with HTTP 409 Conflict; conversely, if a concurrent sibling write F_{sib} (case b, $\mathcal{V}_{F_{\text{sib}}} \parallel \mathcal{V}_F$) is revoked pre-wire during Phase 2, F_{sib} never takes effect and F is immediately unblocked and re-evaluated against all remaining active overlapping writes under Rule 1 (remaining in `SAFE_READ_WAIT` if another overlapping F_{prior} or F'_{sib} is still active, or dispatching as `IMMEDIATE` once no blocking write remains). Furthermore, on sidecar-fronted links, the egress gateway merges the sequence coordinate $\text{Seq}_{\text{dest},F_{\text{sib}}}[d, a_{\text{sib}}]$ of any overlapping chained or recently released sibling write (within $\Delta_{\text{jitter_hold}}$) into $\text{Dep}_{d,F}$, and any read F with non-empty $\text{Dep}_{d,F}$ attaches $\langle \text{Seq}_{\text{dest},F} = 0, \text{Dep}_{d,F} \rangle$ in `X-Agent-Dest-Seq` so the ingress sidecar holds F until $\forall a_k \in \text{dom}(\text{Dep}_{d,F})$, $\text{Seq}_{\text{loc},d}[a_k] \geq \text{Dep}_{d,F}[a_k]$, preventing WAN jitter from reordering a post-release read ahead of an in-flight released write.
2) **Phase 1 — Speculative Epoch Staging (Δ_{stage}):** If `F.is_mutation` = true and F depends on an active upstream speculative reasoning or policy-verification branch E (duration 200–5,000 ms), the L7 filter holds F in user-space staging memory (`STAGE_HELD`) while upstream generation and parallel policy verification execute concurrently. Once upstream epoch E completes verification and commits its decision at verifier TrueTime interval $T_E = [t_{E,\text{earliest}}, t_{E,\text{latest}}]$ (with verifier uncertainty $\epsilon_{\text{verifier}}$), the epoch broadcasts an intra-cluster settlement token $\mathcal{W}(T_E)$ containing $t^*_{E,\text{latest}} = \max(t_{E,\text{latest}}, t_{\text{max},F})$, and the gateway computes the frame's monotonic commit upper bound $t_{F,\text{latest}}$ in $\mathcal{O}(|\mathcal{K}(F)|)$ time by incorporating the per-key settlement floor $t_{\text{key_floor}}[d, k] \in \mathbb{N}_0$ across all $k \in \mathcal{K}(F)$, the destination-wide wildcard floor $t_{\text{key_floor}}[d, \star]$, and—whenever $\star \in \mathcal{K}(F)$—the $\mathcal{O}(1)$ running destination summary watermark $t_{\text{max_all_keys}}[d]$ across all tracked

keys at d:

$$t_{F,\text{latest}} = \max\Big(t^*_{E,\text{latest}},\, t_{\text{key_floor}}[d,\star],\; \max_{k\in\mathcal{K}(F)} t_{\text{key_floor}}[d,k],\, \mathbb{I}(\star\in\mathcal{K}(F))\cdot t_{\text{max_all_keys}}[d]\Big), \tag{5}$$

immediately advancing $t_{\text{key_floor}}[d,k] \leftarrow \max(t_{\text{key_floor}}[d,k], t_{F,\text{latest}} + 1\text{ ns})$ for all $k \in \mathcal{K}(F)$ as well as $t_{\text{max_all_keys}}[d] \leftarrow \max(t_{\text{max_all_keys}}[d], t_{F,\text{latest}} + 1\text{ ns})$ upon Phase 1 settlement (and replicating the updated floor across distributed PTP sidecars in the settlement token while enforcing a minimum elapsed Phase 2 hold of at least $\Delta_{\text{abort_prop}}$ after local settlement token broadcast so that cross-sidecar replication lag and any dynamic Phase 2 read-floor updates complete before the sibling bitmap `b` is finalized at Phase 2 release).

3) **Phase 2 — TrueTime Commit-Wait Serialization (Δ_{wait}):** Upon receiving the settlement token for epoch E, the L7 egress pacer transitions F to `PACED_HELD` and halts wire transmission until its local TrueTime lower bound $t_{\text{now,earliest}} = t_{\text{now,gateway}} - \epsilon_{\text{gateway}}$ strictly surpasses $t_{F,\text{latest}} + \Delta_{\text{abort_prop}}$ (enforcing monotonic same-agent Phase 2 release in $(t_{F,\text{latest}}, c_{\text{tent}})$ order so an earlier-staged frame from a_s is never overtaken in $\text{Seq}_{\text{dest}}[d, a_s]$ assignment by a later-staged frame from a_s), where $\Delta_{\text{abort_prop}}$ is the upper bound on intra-cluster control-plane abort propagation delay from any sibling verification node ($\Delta_{\text{abort_prop}} \le 0.5$ ms over intra-cluster gRPC, or 0 ms under co-located quorum settlement):

$$\begin{aligned}\Delta_{\text{wait}} &= \max(0,\, t_{F,\text{latest}} + \Delta_{\text{abort_prop}} - t_{\text{now,earliest}})\,,\\ \mathbb{E}[\Delta_{\text{wait}}] &\approx \max\big(0,\, \mathbb{E}[\Delta_{\text{floor}}] + 2\epsilon_{\text{eff}} + \Delta_{\text{abort_prop}} - \bar{\Delta}_{\text{ipc}}\big)\end{aligned} \tag{6}$$

where the first line is the **normative runtime barrier** evaluated by polling local TrueTime hardware (since under signed local clock offsets $\delta_u \in [-\epsilon_u, +\epsilon_u]$, physical IPC transit $\Delta_{\text{real_ipc}} \triangleq t_{\text{real,now}} - t_{\text{real},E}$, and combined causal/key floor $\Delta_{\text{floor}} = t_{F,\text{latest}} - t_{E,\text{latest}} \ge 0$, the instantaneous wait equals $\max(0, \Delta_{\text{floor}} + 2\epsilon_{\text{eff}} + (\delta_{\text{verifier}} - \delta_{\text{gateway}}) + \Delta_{\text{abort_prop}} - \Delta_{\text{real_ipc}})$), and the second line gives the tight Jensen's inequality approximation ($\mathbb{E}[\max(0,X)] \approx \max(0, \mathbb{E}[X])$, exact to $< 0.4\%$ zero-clamping) under zero-mean clock offsets ($\mathbb{E}[\delta_{\text{verifier}} - \delta_{\text{gateway}}] = 0$), reducing to $\max(0, 2\epsilon_{\text{eff}} + \Delta_{\text{abort_prop}} - \bar{\Delta}_{\text{ipc}})$ whenever $\Delta_{\text{floor}} = 0$, where $\bar{\Delta}_{\text{ipc}} \triangleq \mathbb{E}[\Delta_{\text{real_ipc}}] = \mathbb{E}[t_{\text{now,gateway}} - t_E]$ denotes expected IPC transit relative to the raw settlement midpoint $t_E \triangleq t_{E,\text{latest}} - \epsilon_{\text{verifier}}$ (which coincides with the floor-adjusted midpoint $t^*_F \triangleq t_{F,\text{latest}} - \epsilon_{\text{verifier}}$ when $\Delta_{\text{floor}} = 0$). A causal or policy abort emitted by a sibling verifier at TrueTime interval $T_{\text{abort}} = [t_{\text{abort,earliest}}, t_{\text{abort,latest}}]$ that either explicitly targets epoch E or preempts a conflicting resource key $k \in \mathcal{K}(F)$ whose physical emission satisfies $t_{\text{real,abort}} \le t_{F,\text{latest}}$ (which implies the conservative interval-overlap condition $t_{\text{abort,earliest}} \le t_{F,\text{latest}}$ since $t_{\text{abort,earliest}} \le t_{\text{real,abort}}$ by Eq. (2)) is deterministically guaranteed to arrive before release: waiting until $t_{\text{now,earliest}} \ge t_{F,\text{latest}} + \Delta_{\text{abort_prop}}$ ensures by Eq. (2) that $t_{\text{real}} \ge t_{F,\text{latest}} + \Delta_{\text{abort_prop}} \ge t_{\text{real,abort}} + \Delta_{\text{abort_prop}}$, so any abort physically emitted at $t_{\text{real,abort}} \le t_{F,\text{latest}}$ has both been emitted and physically arrived at the egress pacer before F is released (whereas aborts physically emitted slightly later in $(t_{F,\text{latest}}, t_{F,\text{latest}} + 2\epsilon_{\text{abort}}]$ that still satisfy $t_{\text{abort,earliest}} \le t_{F,\text{latest}}$ are intercepted opportunistically if they arrive prior to Δ_{wait} expiration, or deterministically if the barrier adds $+2\epsilon_{\text{abort}}$).

4) **Pre-Wire In-Flight Revocation:** If epoch E aborts during Phase 1 (Δ_{stage}) or if a cross-node policy abort targeting E or conflicting on $\mathcal{K}(F)$ with $t_{\text{abort,earliest}} \le t_{F,\text{latest}}$ arrives within the Phase 2 commit-wait window (Δ_{wait}), the unencoded frame F is immediately purged from the user-space L7 queue before any bytes enter the HPACK encoder, TLS record layer, or kernel TCP socket buffer. The gateway returns a deterministic HTTP 409 Conflict response to the calling agent, ensuring **zero mutating payload bytes** reach the downstream SaaS endpoint.

D. Ingress Causal Re-Sequencing Barrier Against WAN Jitter

While sender-side commit-wait pacing orders departure timestamps in t_{real}, public WAN transit jitter (± 10 ms to 60 ms) can reorder packets in flight. For endpoints fronted by an Agent-Chronos ingress sidecar or enterprise API gateway (e.g., fronting SAP S/4HANA OData/RFC connectors), the ingress filter maintains per-destination applied sequence watermarks $\text{Seq}_{\text{loc},d}$ and buffers any arriving mutating frame F from sender a_s until $\text{Seq}_{\text{loc},d}[a_s] = \text{Seq}_{\text{dest},F}[d, a_s] - 1$ and $\forall a_k \in \text{dom}(\text{Dep}_{d,F})$, $\text{Seq}_{\text{loc},d}[a_k] \ge \text{Dep}_{d,F}[a_k]$ (and similarly holds any dependent read carrying $\text{Dep}_{d,F}$ until $\forall a_k \in \text{dom}(\text{Dep}_{d,F})$, $\text{Seq}_{\text{loc},d}[a_k] \ge \text{Dep}_{d,F}[a_k]$; note that because $\text{Seq}_{\text{dest}}[d, a_s]$ is scoped per agent a_s and each agent a_s attaches to a single co-located local PTP sidecar—or shares a consistent logical gateway shard—different agents $a_{s1} \neq a_{s2}$ on distinct PTP sidecars maintain independent sequence counters $\text{Seq}_{\text{dest}}[d, a_{s1}]$ and $\text{Seq}_{\text{dest}}[d, a_{s2}]$ without cross-sidecar sequence contention). For concurrent sibling writes ($\mathcal{V}_1 \parallel \mathcal{V}_2$) where F_1 (from a_{s1}) precedes F_2 (from a_{s2}) in deterministic ($t_{F,\text{latest}}$, AgentID) order ($t_{F_1,\text{latest}} < t_{F_2,\text{latest}}$, or equality broken by $\text{AgentID}(a_{s1}) < \text{AgentID}(a_{s2})$), the monotonic per-key settlement floor $t_{\text{key_floor}}[d,k]$ in Eq. (5) guarantees that any sibling settling Phase 1 after F_1 receives $t_{F_2,\text{latest}} > t_{F_1,\text{latest}}$. Upon Phase 2 release of the later-ordered write F_2, the egress gateway finalizes the 16-bit sibling fork bitmap `b` in `X-Agent-Causal-Fence-Vector`, setting bits *strictly* for concurrent sibling agents a_{s1} that have already settled Phase 1 with an earlier ($t_{F_1,\text{latest}}$, AgentID) order and overlapping resource keys ($\mathcal{K}(F_1) \cap \mathcal{K}(F_2) \neq \emptyset$) targeting d. Because monotonic same-agent Phase 2 release guarantees that any multiple overlapping writes from a_{s1} release in ticket order, F_2 only needs to reference the highest preceding overlapping ticket $c_{\text{tent},1}$ per flagged sibling a_{s1}, serializing the per-sibling coordinate map $\text{SibCoord}_{d,F_2} : a_{s1} \mapsto \langle t_{F_1,\text{latest}}, c_{\text{tent},1}, s_{\text{tomb}}\rangle$ in `X-Agent-Dest-Seq` with primary matching key $\langle a_{s1}, c_{\text{tent},1}\rangle$ (where $s_{\text{tomb}} = \text{Seq}_{\text{dest},F_1}[d, a_{s1}] > 0$ when F_1 has already completed Phase 2 release prior to F_2; whereas if F_1 is still in `PACED_HELD` on a peer PTP sidecar so $s_{\text{tomb}} = 0$ at F_2's departure, a_{s1}'s sidecar detects F_2's overlapping settlement token via intra-cluster gossip, marks F_1 as watched, and emits a

lightweight control-plane SIBLING_SEQ_BIND frame carrying $\langle a_{s1}, t_{F_1,\text{latest}}, c_{\text{tent},1}, s_{\text{tomb}}\rangle$ directly to destination d's ingress sidecar immediately upon F_1's Phase 2 release, or emits SIBLING_CLEAR carrying $\langle a_{s1}, t_{F_1,\text{latest}}, c_{\text{tent},1}\rangle$ if F_1 is revoked during Phase 2; the ingress sidecar caches both in a bounded TTL set [$2\Delta_{\text{jitter_hold}}$, i.e., 120 ms at $\Delta_{\text{jitter_hold}} =$ 60 ms] keyed by $\langle a_{s1}, c_{\text{tent},1}\rangle$ so that even if WAN jitter delivers SIBLING_CLEAR or SIBLING_SEQ_BIND *before* F_2 arrives, F_2 immediately recognizes F_1's bound sequence or cleared state, while aborted, later-settled, read-only, or disjoint-key sibling branches never set bits in F_2's bitmap). Upon ingress arrival of F_2 once $t_{\text{now,earliest}} \geq t_{F_2,\text{latest}}$, the ingress sidecar initiates a bounded WAN jitter countdown $\Delta_{\text{jitter_hold}}$ *if and only if* F_2's bitmap $\mathtt{b}$ contains at least one bit for an earlier-ordered sibling write F_1 that has neither been applied at the ingress sidecar nor recorded in the SIBLING_CLEAR TTL cache; this hold terminates **immediately** (0 ms further wait) as soon as all flagged earlier sibling writes F_1 arrive and are applied (or cleared), waiting the full $\Delta_{\text{jitter_hold}} = (t_{\text{wan,max}} - t_{\text{wan,min}}) + 2\epsilon_{\text{gateway_diff}} + \Delta_{\text{abort_prop}}$ (where $\epsilon_{\text{gateway_diff}} \triangleq \epsilon_{\text{gateway},1} \leq \epsilon_{\text{PTP,max}}$ bounds the worst-case physical Phase 2 departure skew when peer sidecar 1 runs at $-\epsilon_{\text{gateway},1}$ while sidecar 2 runs at $+\epsilon_{\text{gateway},2}$, and $\epsilon_{\text{gateway_diff}} = 0$ when F_1 and F_2 share the same physical gateway clock reference $\delta_1 = \delta_2$, reducing $\Delta_{\text{jitter_hold}}$ to $t_{\text{wan,max}} - t_{\text{wan,min}} = 60$ ms under a shared regional egress gateway) only when F_1 is pathologically delayed. Conflicting writes are applied in deterministic ($t_{F,\text{latest}}$, AgentID) order once $t_{\text{now,earliest}} \geq t_{F,\text{latest}}$. If a pathologically delayed earlier sibling write F_1 that was released from Phase 2 ($s_{\text{tomb}} = \text{Seq}_{\text{dest},F_1}[d, a_{s1}] > 0$, provided either in F_2 or via SIBLING_SEQ_BIND) fails to arrive before F_2's $\Delta_{\text{jitter_hold}}$ expires and F_2 is applied with $t_{F_1,\text{latest}} < t_{F_2,\text{latest}}$, the ingress sidecar proactively advances $\text{Seq}_{\text{loc},d}[a_{s1}]$ to s_{tomb} upon $\Delta_{\text{jitter_hold}}$ expiration while recording $\langle a_{s1}, t_{F_1,\text{latest}}, c_{\text{tent},1}, s_{\text{tomb}}\rangle$ in a late-rejection tombstone set (so subsequent disjoint-key frames from a_{s1} with $\text{Seq}_{\text{dest}} = s_{\text{tomb}} + 1$ never stall between 60 ms and 180 ms, and when the delayed F_1 eventually arrives it is deterministically rejected with HTTP 409 Stale Sibling Timestamp; if $s_{\text{tomb}} = 0$ at $\Delta_{\text{jitter_hold}}$ expiration because neither F_1 nor SIBLING_SEQ_BIND has arrived, the ingress sidecar records the unbound tombstone $\langle a_{s1}, t_{F_1,\text{latest}}, c_{\text{tent},1}\rangle$ and emits a control-plane TOMBSTONE_FENCE $\langle d, a_{s1}, c_{\text{tent},1}\rangle$ notification to a_{s1}'s egress sidecar, which fences F_1 and replies with a TOMBSTONE_ACK $\langle d, a_{s1}, c_{\text{tent},1}, s_{\text{tomb}}\rangle$ frame binding s_{tomb}—also emitted automatically if a released HTTP/2 stream resets mid-flight via RST_STREAM—so the ingress sidecar immediately skips s_{tomb} in $\text{Seq}_{\text{loc},d}[a_{s1}]$). Each agent initializes $\text{AckTomb}_{d,i} : \text{AgentID} \to \mathbb{N}_0$ to 0 and advances $\text{AckTomb}_{d,i}[a_{s1}] \leftarrow \max(\text{AckTomb}_{d,i}[a_{s1}], s_{\text{tomb}})$ upon processing a 409 tombstone rejection or TOMBSTONE_FENCE. Upon tombstoning $s_{\text{tomb}} > 0$, the ingress sidecar cascades HTTP 409 aborts strictly to *causally dependent* unapplied in-flight frames ($\mathcal{V}_{F_1} \prec_{\mathcal{V}} \mathcal{V}_F$, where a_{s1} scopes $\mathcal{V}_{F_1}$ to $\mathcal{K}(F_1)$ so concurrent disjoint-key frames F_1' with $\mathcal{K}(F_1') \cap \mathcal{K}(F_1) = \emptyset$ satisfy $\mathcal{V}_{F_1} \not\prec_{\mathcal{V}} \mathcal{V}_{F_1'}$ and are never cascade-aborted) carrying $\text{Dep}_{d,F}[a_{s1}] \geq s_{\text{tomb}}$ whose acknowledged-tombstone watermark in X-Agent-Dest-Seq satisfies $\text{AckTomb}_{d,F}[a_{s1}] < s_{\text{tomb}}$, while unblocking concurrent sibling reads ($\mathcal{V}_{F_1} \parallel \mathcal{V}_F$) once $\text{Seq}_{\text{loc},d}[a_{s1}] \geq s_{\text{tomb}}$ so they read the state committed by F_2 (and permitting post-409 recovery writes synthesized after the agent processes the rejection, which carry $\text{AckTomb}_{d,F}[a_{s1}] \geq s_{\text{tomb}}$). For unmodified external third-party SaaS APIs without an ingress sidecar, the sender-side gateway acquires per-resource pipeline locks in $\mathcal{K}(F)$ (with intention-lock IS/IX vs. exclusive root-lock compatibility on $\{\star\}$ as specified in Section II-C) upon Phase 2 transition to RELEASED in deterministic ($t_{F,\text{latest}}$, AgentID) order (and across multi-key sets in canonical lexicographical key order to prevent deadlocks), holds any dependent write or read F_2 ($\mathcal{K}(F_1) \cap \mathcal{K}(F_2) \neq \emptyset$) until the HTTP 2xx acknowledgment of F_1 returns, and pairs post-transmission HTTP 5xx faults with compensating sagas [4]. To tolerate node failures of the shared logical egress serialization gateway during in-flight SaaS transmission or saga recovery, per-resource lock leases and pre-dispatch compensating saga intent records $\langle d, \mathcal{K}(F_1), \text{IdempKey}(F_1), \text{CompSpec}(F_1), \text{epoch}_{\text{fence}}\rangle$ are synchronously replicated to an intra-cluster Raft/Spanner-backed lock-lease table prior to wire dispatch, while the primary egress shard enforces a local kernel socket send/ACK timeout SO_SNDTIMEO = TCP_USER_TIMEOUT = 150 ms $< \tau_{\text{lease}} - 2\epsilon_{\text{eff}}$ and verifies remaining TrueTime lease validity $t_{\text{lease_exp}} - t_{\text{now,latest}} >$ SO_SNDTIMEO before writing to the socket (arming an abortive SO_LINGER {l_onoff=1, l_linger=0} TCP_RST close and cloud SDN/VIP source-packet fence if the lease expires prior to HTTP 2xx completion so unsent kernel sk_buff segments cannot drain after lease loss): if the primary egress gateway shard crashes or its lease expires ($\tau_{\text{lease}} = 250$ ms), the standby egress shard waits $\tau_{\text{lease}} + 2\epsilon_{\text{eff}}$ (accounting for worst-case cross-node clock uncertainty), increments the monotonic fencing epoch $\text{epoch}_{\text{fence}} \leftarrow \text{epoch}_{\text{fence}} + 1$, fence-isolates the failed gateway, resolves $\text{IdempKey}(F_1)$ on the external SaaS API (either invoking a provider cancellation/void endpoint or replaying F_1 under the identical $\text{IdempKey}(F_1)$ to deterministically settle F_1's state and prevent any late-draining packet from executing uncompensated if not already committed), and drives $\text{CompSpec}(F_1)$ rollback to terminal completion before releasing the resource lock to queued frame F_2.

III. Wire Protocol and Observability Invariants

Internal pre-wire gateway queue states (STAGE_HELD, PACED_HELD) resolve to four terminal wire dispositions ({IMMEDIATE, SAFE_READ_WAIT, RELEASED, REVOKED}) prior to wire emission (while sidecar-to-sidecar control notifications for Phase 2 sibling revocations, sequence binding, and post-release stream resets use zero-payload HTTP/2 HEADERS-only control frames with END_STREAM=1 and X-Agent-Pacing-Status set to SIBLING_CLEAR, SIBLING_SEQ_BIND, TOMBSTONE_FENCE, or TOMBSTONE_ACK, which are always dispatched on dedicated ephemeral HTTP/2 control stream IDs multiplexed over the h2 connection—never on an active bidirectional MCP/A2A stream ID—and are terminated and consumed strictly at the Chronos sidecar without forwarding any request to the backend ERP/SaaS service). Each speculative unary or client-streaming gRPC/MCP mutation either opens a dedicated multiplexed HTTP/2 stream whose initial HEADERS and request DATA frames

are held in user-space until Phase 2 completes, or—on long-lived bidirectional MCP/A2A streams—holds individual JSON-RPC request `DATA` frames in user-space `STAGE_HELD/PACED_HELD` queues prior to wire emission (carrying per-call `X-Agent-*` settlement fields inside the L7 JSON-RPC `_meta` envelope upon Phase 2 release, which the Chronos ingress sidecar's L7 JSON-RPC framer inspects and strips/verifies per message identically to unary HTTP/2 headers before forwarding to the local MCP/A2A server, while session-closing HTTP/2 `END_STREAM` trailers emit a cumulative session audit digest), strictly preserving the zero-byte wire invariant upon revocation without prematurely half-closing active bidirectional streams. Agent-Chronos injects five `X-Agent-*` settlement, pacing, and sequence HTTP/2 headers:

- `X-Agent-TrueTime-Window`: Serialized settlement tuple $\langle t_{E,\text{earliest}}, t_{F,\text{latest}}, \epsilon_{\text{eff}}[, \epsilon_{\text{verifier}}]\rangle$ in integer nanoseconds for settled frames (including $\epsilon_{\text{verifier}}$ whenever $\epsilon_{\text{verifier}} \neq \epsilon_{\text{eff}}$, and defaulting to ϵ_{eff} on single-node `IMMEDIATE` reads), where $t_{F,\text{latest}}$ is given by Eq. (5), $\epsilon_{\text{eff}} = \lceil(\epsilon_{\text{verifier}} + \epsilon_{\text{gateway}})/2\rceil$, and receivers verify $t_{F,\text{latest}} - t_{E,\text{earliest}} \geq 2\epsilon_{\text{verifier}} > 0$ on `RELEASED` frames (and exact equality $t_{\text{now},i,\text{latest}} - t_{\text{now},i,\text{earliest}} = 2\epsilon_{\text{gateway}}$ on `IMMEDIATE` reads).
- `X-Agent-Causal-Fence-Vector`: Compact base64/JSON serialization ($\leq$ 256 B) of settled $\mathcal{V}_F = \langle \mathbf{C}_F, t_{F,\text{latest}}\rangle$ and 16-bit active preceding sibling fork bitmap `b`.
- `X-Agent-Pacing-Status`: Terminal disposition in {`IMMEDIATE`, `SAFE_READ_WAIT`, `RELEASED`, `REVOKED`} (plus sidecar-internal control dispositions `SIBLING_CLEAR`, `SIBLING_SEQ_BIND`, `TOMBSTONE_FENCE`, `TOMBSTONE_ACK`).
- `X-Agent-Pacing-Delay-Ms`: Post-settlement gateway hold duration in milliseconds (Δ_{wait} for `RELEASED` mutations, chained wait for `SAFE_READ_WAIT` reads, and elapsed Phase 2 hold prior to abort for `REVOKED` frames; Phase 1 Δ_{stage} is exported in OpenTelemetry attribute `chronos.stage_delay_ms`).
- `X-Agent-Dest-Seq`: Per-destination transport sequence coordinate and dependency tuple $\langle \text{Seq}_{\text{dest},F}[d, a_s], \text{Dep}_{d,F}, \text{AckTomb}_{d,F}, \text{SibCoord}_{d,F}\rangle$ attached to mutating calls and dependent reads (with $\text{Seq}_{\text{dest},F} = 0$ on dependent reads) for gap-free ingress FIFO delivery, sibling tombstone binding, and cascade-abort scoping.

Dual-Plane Observability of Revoked Frames: When a frame is purged via Pre-Wire In-Flight Revocation, zero mutating bytes are transmitted to the external SaaS target. Instead, `X-Agent-Pacing-Status: REVOKED` and `X-Agent-Pacing-Delay-Ms` are emitted on (i) the synchronous HTTP 409 response wire back to the upstream orchestrator and (ii) a payload-free signed telemetry trailer exported to the cross-organization OpenTelemetry audit sink, ensuring 100% protocol auditability without leaking aborted payloads to third-party APIs.

IV. Empirical Evaluation

A. Experimental Testbed and Workload Methodology

We evaluated Agent-Chronos on an emulated cross-cloud HTTP/2 Envoy + FastMCP testbed modeling multi-agent financial ledger and procurement workflows (adapted from τ-bench [15] tool-invocation schemas) between Google Cloud Vertex AI and an SAP S/4HANA ERP gateway across $N = 1{,}000$ multi-agent transactions per seed (4,200 total tool frames per run: 2,520 [60%] mutating frames and 1,680 [40%] read-only queries over a $|\mathcal{K}| = 64$ resource key-space under Zipf($s = 1.15$) contention, evaluated across 5 independent random seeds [5,000 total transactions per baseline] with a controlled fault-injection budget of 148 aborted transactions per seed [14.80%, 0.0 across-seed SD by design to isolate protocol behavior under a fixed fault rate, comprising 372 aborted mutating frames per seed across the 148 aborted transactions, or $\approx$ 2.51 mutating frames/tx] and stochastic network/clock draws across seeds). The 6-agent hierarchical DAG topology comprises 1 root orchestrator, 2 tier-1 domain planners, and 3 tier-2 execution subagents (depth 3, max branching factor 2, baseline $t_{\text{exec}} = 650.0 \pm 161.2$ ms mean tool/agent execution path, which decomposes into $t_{\text{prep}} = 560.0 \pm 159.3$ ms of upstream agent reasoning, tool synthesis, and staging plus $t_{\text{rtt}} = 90.0 \pm 24.5$ ms of post-release final commit WAN RTT), where each transaction dispatches 1 to 2 exploratory read queries and 2 to 3 mutating tool calls (mean 4.20 frames/transaction, with 46.6% of transactions containing a two-step dependent same-key write chain [an intermediate preparatory mutation followed by a terminal settlement mutation] and 5.8% [97.4/1,680] of reads issuing post-settlement verification queries during an active Phase 2 `PACED_HELD` window on the same key), where intermediate non-terminal mutating writes inherit the speculative epoch E of their enclosing transaction branch so all 2–3 mutating frames per transaction are held in user-space `STAGE_HELD` during Phase 1 and settled together upon completion of epoch E (releasing in causal sequence order on sidecar-fronted links, or incurring one additional 90.0 ms sequential pipeline RTT prior to the terminal write across the 46.6% chained transactions [0.466×90.0 ms $= 42.0$ ms amortized per transaction] on unmodified external SaaS endpoints). Network links were configured with Linux `tc netem` around a 90 ms base RTT (45 ms one-way base delay with ± 30 ms independent uniform per-direction jitter, yielding Triangular(-60 ms, 0 ms, $+60$ ms) RTT jitter, strictly positive one-way transit $\in [15, 75]$ ms, and ingress sibling hold bound $\Delta_{\text{jitter_hold}} = 60$ ms matching the $75 - 15 = 60$ ms max one-way spread; co-located regional verifiers use $\Delta_{\text{wan_pre}} = 0$ ms), independent verifier and egress gateway TrueTime drift bounds $\epsilon_{\text{verifier}}, \epsilon_{\text{gateway}}$ i.i.d. $\sim$ Uniform(1.0 ms, 7.0 ms) (whose sum $2\epsilon_{\text{eff}} = \epsilon_{\text{verifier}} + \epsilon_{\text{gateway}} \sim$ Triangular(2.0 ms, 8.0 ms, 14.0 ms) has mean 8.00 ms and theoretical SD $\sqrt{6} \approx 2.45$ ms, with common-mode regional master reference $\delta_{\text{verifier}} - \delta_{\text{gateway}} = 0$ ms [$\sigma^2_{\Delta\delta} = 0$ ms^2] in Federated Egress Gateway Mode), upstream speculative verification durations Δ_{stage} clamped to $[200, 5{,}000]$ ms from LogNormal($\mu = 6.2, \sigma = 0.55$) (theoretical unclamped median 492.7 ms, 5-seed empirical median 491.4 ms, and clamped arithmetic mean 572.7 ± 336.8 ms), and sibling verifier completion offsets following

$\Delta t_{\text{sib}} \sim \text{Normal}(\mu = 0\text{ ms}, \sigma = 22\text{ ms})$ around T_E—yielding a stochastic clock/network boundary split across the 148.0 aborts of 119.0 ± 1.1 (80.41%, 5-seed mean $\pm$ SD) aborting during Phase 1 concurrent verification and 29.0 ± 1.1 (19.59%) arriving within the $2\epsilon_{\text{eff}}$ cross-node clock-skew window during Phase 2. Because net intra-cluster settlement token transit minus abort margin ($X = \Delta_{\text{real_ipc}} - \Delta_{\text{abort_prop}} \sim \text{LogNormal}(\mu = 0.72, \sigma = 0.18)$ ms, mean 2.09 ± 0.38 ms with Gaussian-copula load correlation $\rho = 0.42$ against $2\epsilon_{\text{eff}}$, co-located reference analytical variance $6.00 + 0.146 - 2(0.42)(2.449)(0.382) = 5.36\text{ ms}^2$ [SD = 2.32 ms], and 0.0% zero-clamping since $\min(2\epsilon_{\text{eff}}) = 2.0$ ms) elapses before $t_{\text{now,earliest}}$ is queried, the residual Phase 2 wait Δ_{wait} pooled over the $2{,}520 - 372 = 2{,}148$ non-aborted committed mutating frames per seed averages 5.91 ± 2.32 ms ($p_{50} = 5.90$ ms, $p_{95} = 9.80$ ms, $p_{99} = 10.84$ ms; when sampling independent non-zero local clock offsets δ_u with $\sigma_{\Delta\delta} = 0.50$ ms [$\sigma^2_{\Delta\delta} = 0.25\text{ ms}^2$] across decoupled references, total analytical variance increases to $5.36 + 0.25 = 5.61\text{ ms}^2$ [SD = 2.37 ms], yielding $\Delta_{\text{wait}} = 5.91 \pm 2.37$ ms with $p_{95} = 9.81$ ms, $p_{99} = 10.96$ ms). Transaction-level causal inversions are counted out of $N = 1{,}000$ transactions when any mutating or dependent frame arrives at the destination ERP ingress boundary ahead of a causally preceding frame ($\mathcal{V}_1 \prec_{\mathcal{V}} \mathcal{V}_2$) or out of physical commit order across concurrent conflicting writes.

B. Comparative Results and Component Ablations

Table I compares Agent-Chronos against four baselines: (1) an **Unmitigated Baseline Swarm**, (2) **Application-Layer Compensating Sagas / Post-Facto Undo** [4], [13], [14] (where compensation RPCs drawn from $\text{LogNormal}(\mu = 4.90, \sigma = 0.35)$ ms [mean 142.53 ± 51.59 ms, conditional $p_{50}/p_{95}/p_{99} = 135.39/233.63/300.82$ ms] are dispatched across all 148.0 aborted transactions per seed, succeeding on 106.0 ± 1.6 [71.62%] reversible writes and failing with an external settlement-lock error after the RPC round-trip on 42.0 ± 1.6 [28.38%] non-reversible writes), (3) a software-only **NTP Hybrid Logical Clock (HLC) Barrier** [8] enforcing dynamic $\Delta_{\text{wait,NTP}} = \max(0, 2\hat{\epsilon}_{\text{NTP}} + \Delta_{\text{abort_prop}} - \bar{\Delta}_{\text{ipc}})$ over local NTP daemon error estimates $\hat{\epsilon}_{\text{NTP}} \sim \text{Normal}(\mu = 35\text{ ms}, \sigma = 8\text{ ms})$ plus a 5% Bernoulli spike mixture $\text{Uniform}(40, 85)$ ms (where half of spikes are reflected in $\hat{\epsilon}_{\text{NTP}}$—raising wait SD to 19.20 ms and preventing abort leaks—and half are uncompensated physical offset spikes on sibling branches that exceed the 25 ms scalar HLC ingress watermark hold window, causing 21.0 ± 1.9 [2.10%] sibling causal inversions and 1.2% of post-settlement dependent reads observing stale state), and (4) a **Chronos Vector-Only Ablation** (removing Phase 2 TrueTime commit-wait pacing while retaining Phase 1 staging, logical vector clocks, and per-destination sequence fencing—serving as an architectural proxy for logical pre-commit staging runtimes such as Atomix [18], Sherlock [17], Speculative Actions [16], and ALAS [19] when deployed across *decentralized cross-node verification topologies*).

In the Unmitigated Baseline, the 195.0 ± 4.2 (19.50%) destination causal inversions decompose into 148.0 ± 0.0 transactions where an unrevoked speculative mutation arrived ahead of its subsequent abort/recovery branch plus 47.0 ± 4.2 pure WAN packet-reordering inversions across valid multi-frame transactions. Ablating TrueTime commit-wait (**Chronos Vector-Only Ablation**) intercepts the 119.0 ± 1.1 (80.41%) Phase 1 aborts before wire release, but leaks the 29.0 ± 1.1 (2.90%) cross-node race aborts arriving within the $2\epsilon_{\text{eff}}$ physical clock-skew window; because each leaked mutation executes ahead of the orchestrator's subsequent recovery write on the same resource key—and because ablating TrueTime physical bounds forces the ingress sidecar to fall back to arrival-order tie-breaking on vector-concurrent colliding sibling writes ($\mathcal{V}_1 \parallel \mathcal{V}_2$, 5.0 ± 0.9 collisions/run under Zipf(1.15)), Vector-Only incurs $29.0 + 5.0 = 34.0 \pm 1.5$ (3.40%) destination inversions. Note that ingress jitter re-sequencing hold latency (8.42 ± 4.90 ms, which includes $+0.03$ ms amortized from sibling-fork holds across the $5.0/2{,}148$ colliding committed frames) matches between Vector-Only and Full Agent-Chronos on the 90 ms WAN link because minimum one-way transit (15 ms) already exceeds $2\epsilon_{\text{eff}} \leq 14$ ms upon arrival ($t_{\text{now,earliest}} \geq t_{F,\text{latest}}$); by contrast, in an auxiliary low-latency intra-region metro interconnect experiment (4 ms base RTT, one-way transit $[1, 3]$ ms $< 2\epsilon_{\text{eff}}$), the ingress physical-time barrier ($t_{\text{now,earliest}} \geq t_{F,\text{latest}}$) actively holds arriving frames for 3.92 ± 1.85 ms to preserve 0.00% inversions.

Full **Agent-Chronos** intercepts 100.00% (148.0/148.0) of aborted mutations pre-wire and eliminates 100.00% (195.0/195.0) of causal inversions, with a mean Phase 2 commit-wait of 5.91 ± 2.32 ms ($p_{50} = 5.90$ ms, $p_{95} = 9.80$ ms, $p_{99} = 10.84$ ms; when remote cross-cloud quorum verifiers require $\Delta_{\text{wan_pre}} = 20$ ms, pre-minting quorum wait adds $+18.5 \pm 4.2$ ms) and 0.00 ms delay on independent read-only queries (94.2% bypass immediately; the 83.0 non-aborted post-settlement dependent reads wait 5.40 ± 1.98 ms via `SAFE_READ_WAIT`). End-to-end, compared to non-speculative sequential execution ($T_{\text{seq}} = t_{\text{exec}} + \Delta_{\text{stage}} = 650.0 + 572.7 = 1{,}222.7 \pm 373.3$ ms mean completion, summing the 650.0 ms execution path and 572.7 ms clamped mean verification duration [median 492.7 ms]), Agent-Chronos overlaps upstream agent synthesis/staging ($t_{\text{prep}} = 560.0$ ms) with verification (Δ_{stage}): across the 85.2% non-aborted committed transactions, completion latency is $T_{\text{spec,commit}} = \max(t_{\text{prep}}, \Delta_{\text{stage}}) + \Delta_{\text{wait}} + t_{\text{rtt}} = 703.6 + 5.91 + 90.0 = 799.5 \pm 278.1$ ms (where $\mathbb{E}[\max(t_{\text{prep}}, \Delta_{\text{stage}})] = 703.6 \pm 276.9$ ms is the empirical 5-seed mean of the pointwise maximum between independent draws $t_{\text{prep}} \sim \text{Normal}(560.0, 159.3^2)$ ms and clamped $\Delta_{\text{stage}} \sim \text{LogNormal}(\mu = 6.2, \sigma = 0.55)$ ms, reflecting the $+143.6$ ms upper-tail excess when $\Delta_{\text{stage}} > t_{\text{prep}}$), achieving a net speculative speedup of 423.3 ms (34.6%). Across all 1,000 transactions unconditionally (including the 14.8% aborted branches, which incur $+240.0 \pm 45.0$ ms fallback re-synthesis upon pre-wire revocation, adding $+34.7$ ms unconditional amortized retry overhead), mean completion is 834.2 ± 292.5 ms (an unconditional speedup of 388.5 ms, or 31.8%). When invoking unmodified third-party SaaS APIs without an ingress sidecar via sequential per-resource in-flight pipelining (0% injected HTTP 5xx faults, incurring one 90 ms WAN RTT for the intermediate preparatory write prior to the terminal write across the 46.6% two-write chained transactions: 0.466×90.0 ms $= 42.0$ ms), mean transaction latency increases by $+42.0 \pm 19.4$ ms (841.5 ms committed / 876.2 ms unconditional) with 0.00% phantom

TABLE I
EMPIRICAL BENCHMARK AND ABLATION COMPARISON ACROSS $N = 1{,}000$ CROSS-CLOUD MULTI-AGENT TRANSACTIONS PER SEED (5 SEEDS)[a]

Metric (Basis / Granularity)	Unmitigated Baseline	Compensating Sagas [4], [13], [14]	NTP Software HLC [8]	Chronos Vector-Only (Ablation)	Agent-Chronos (Full)
Transactions Evaluated (Per Seed, $n = 5$ Seeds)	1,000	1,000	1,000	1,000	**1,000**
Phantom Tool Mutations (Tx Count, % of 1,000 Tx)[a]	148.0 ± 0.0 (14.80%)	42.0 ± 1.6 (4.20% uncomp.)	0.0 ± 0.0 (0.00%)	29.0 ± 1.1 (2.90% skew leak)	$\mathbf{0.0 \pm 0.0}$ (**0.00%**)
Causal Inversion Rate (Tx Count, % of 1,000 Tx)[a]	195.0 ± 4.2 (19.50%)	195.0 ± 4.2 (19.50%)	21.0 ± 1.9 (2.10%)	34.0 ± 1.5 (3.40%)	$\mathbf{0.0 \pm 0.0}$ (**0.00%**)
Pre-Wire Aborted Revocations (% of 148 Aborted Tx)[a]	0.0 ± 0.0 (0.00%)	0.0 ± 0.0 (0.00%)	148.0 ± 0.0 (100.00%)	119.0 ± 1.1 (80.41%)	$\mathbf{148.0 \pm 0.0}$ (**100.00%**)
Mean Commit-Wait / Sync Overhead (Δ_{wait}, Per-Frame SD)[a,b]	0.00 ms	0.00 ms ($+142.53 \pm 51.59$ ms undo)[b]	68.40 ± 19.20 ms	0.00 ms	$\mathbf{5.91 \pm 2.32}$ ms
Commit-Wait / Undo Tail Latency ($p_{50}/p_{95}/p_{99}$)[b]	0.00/0.00/0.00 ms	0.00/155.73/220.89 ms (undo)[b]	67.10/98.40/112.60 ms	0.00/0.00/0.00 ms	**5.90**/**9.80**/**10.84** ms
Ingress Jitter Re-Sequencing Hold (Per-Frame SD)[a]	0.00 ms	0.00 ms	11.20 ± 6.40 ms	8.42 ± 4.90 ms	$\mathbf{8.42 \pm 4.90}$ ms
Independent Read-Only Bypass (94.2% of 1,680 Reads)	0.00 ms	0.00 ms	0.00 ms	0.00 ms	**0.00** ms
Dependent Read-Only Wait (`SAFE_READ_WAIT`, 5.8% of Reads)[c]	0.00 ms (stale read)	0.00 ms (stale read)	61.20 ± 17.50 ms	0.00 ms (stale read in $2\epsilon_{\text{eff}}$)[c]	$\mathbf{5.40 \pm 1.98}$ ms
Protocol Auditability	None	Application Logs Only	Partial (Scalar HLC)	Logical Vector Only	**100.00% (Dual-Plane)**

[a]Event-count rows report 5-seed mean $\pm$ across-seed SD ($n = 5$ seeds; total abort budget is fixed at 148.0 ± 0.0 Tx/seed [372 aborted mutating frames/seed]); continuous latency rows report 5-seed mean $\pm$ pooled per-frame SD over the 2,148 committed mutating frames/seed.
[b]Compensating Sagas incur 0.00 ms preventive pre-release wait (contrasted against reactive post-transmission undo: $+142.53 \pm 51.59$ ms conditional rollback [LogNormal($\mu = 4.90$, $\sigma = 0.35$) ms] across the 148 aborted Tx/seed, or $+21.09 \pm 54.37$ ms unconditional across 1,000 Tx), while 0.00/155.73/220.89 ms (undo) reports unconditional rollback percentiles across all 1,000 Tx (85.2% non-aborted at 0.00 ms; conditional $p_{50}/p_{95}/p_{99} = 135.39/233.63/300.82$ ms, where unconditional p_{95}, p_{99} equal conditional $p_{66.2}, p_{93.2}$).
[c]Of the 5.8% (97.4/1,680) post-settlement dependent reads/seed (83.0 non-aborted, 14.4 aborted via HTTP 409), 100% observe stale state under Unmitigated/Saga, 19.6% under Vector-Only ($\Delta_{\text{wait}} = 0$ ms), 1.2% under NTP HLC, and 0.0% under Chronos.

mutations and 0.00% inversions; under a 2% injected post-transmission HTTP 5xx fault rate (20.0 faults per 1,000 Tx) on the unmodified SaaS fallback while holding the sender-side per-resource pipeline lock during saga recovery (0.00% causal inversions), compensating sagas (LogNormal($\mu = 4.90, \sigma = 0.35$) ms) roll back 14.3 ± 0.9 (71.6%) reversible faults (adding $+2.85$ ms unconditional amortized rollback latency) while leaving 5.7 ± 0.9 (0.57% of 1,000 Tx) uncompensated faults on non-reversible endpoints, empirically quantifying the post-transmission SaaS boundary relative to sidecar-fronted pre-wire revocation. Evaluating Hardware PTP Sidecar Mode ($\epsilon_{\text{PTP},u}$ i.i.d. $\sim$ Uniform(1.0 ms, 10.0 ms), $2\epsilon_{\text{eff}} \sim$ Triangular(2.0 ms, 11.0 ms, 20.0 ms)) similarly achieved 0.00% phantom mutations and 0.00% inversions at a mean commit-wait of 8.84 ± 3.12 ms and ingress hold of 8.76 ± 5.04 ms (reflecting the 15–20 ms upper tail of $2\epsilon_{\text{eff}}$), while user-space L7 staging required only 1.8 MB peak memory across 100 concurrent staged frames and < 0.12 ms CPU overhead per frame. Complementing the 5,000-transaction controlled fault-injection benchmark, a live 100-transaction cross-region validation deployed between Google Cloud `us-central1` (Vertex AI ADK orchestrator + Chronos egress sidecar) and `europe-west1` (Chronos ingress sidecar fronting a FastMCP ledger service, measured 88.4 ± 11.2 ms WAN RTT and $\epsilon_{\text{eff}} \approx 4.1$ ms) achieved 0.00% (0/100) phantom mutations and 0.00% (0/100) causal inversions at a mean Phase 2 commit-wait of 6.12 ± 2.45 ms ($p_{99} = 11.10$ ms) and ingress hold of 7.95 ± 4.62 ms. Finally, an auxiliary stress suite confirmed 0.00% phantom mutations and 0.00% inversions under: (i) heavy-tailed Pareto($\alpha = 2.5$) WAN jitter spikes up to 180 ms (where conflicting sibling writes arriving past $\Delta_{\text{jitter_hold}} = 60$ ms are safely rejected with HTTP 409 Stale Sibling Timestamp), (ii) temporally autocorrelated sawtooth GPS holdover (ϵ_u ramping to 12 ms, raising p_{99} commit-wait to 19.40 ms), (iii) 2% injected mid-flight HTTP/2 `RST_STREAM` resets (resolved via tombstone sequence ACKs with zero ingress stalls), and (iv) 16-agent swarm fan-out (121 B unformatted positional / 184 B with optional metadata max raw JSON vector).

V. CONCLUSION AND OPERATIONAL SCOPE

Agent-Chronos establishes a hardware-anchored L7 transport fencing and speculative pacing protocol for cross-cloud multi-agent swarms. By combining two-phase user-space speculative staging, TrueTime commit-wait serialization ($\Delta_{\text{wait}} = \max(0, t_{F,\text{latest}} + \Delta_{\text{abort_prop}} - t_{\text{now,earliest}})$), decoupled swarm-wide causal vectors and per-destination sequence coordinates, and ingress jitter re-sequencing, Agent-Chronos eliminates phantom tool mutations and causal inversions across Chronos-enabled sidecar gateways while delivering a 423.3 ms (34.6%) net speculative latency reduction per committed transaction (388.5 ms, or 31.8%, unconditionally across all transactions) without holding distributed database locks. Functionally, Agent-Chronos guarantees strict client-side egress linearizability and end-to-end causal delivery across sidecar-fronted endpoints rather than internal database strict serializability; when invoking unmodified third-party SaaS APIs that lack an ingress sidecar, the gateway suppresses tombstone sequence ACKs, routes concurrent sibling writes ($\mathcal{V}_1 \parallel \mathcal{V}_2$) through a shared logical egress gateway, and falls back to sequential per-resource in-flight pipelining ($+42.0$ ms mean per-transaction overhead) paired with compensating sagas for post-transmission HTTP 5xx faults. Operational deployment assumes access to TrueTime, local `/dev/ptp0` hardware clocks, or a federated Chronos egress gateway, alongside bounded active swarm fan-out ($|\text{AgentID}| \leq 16$, or hierarchical cluster watermarks for > 100-agent meshes). Future work will extend Agent-Chronos to dynamic multi-tier cluster reconfiguration and hardware NIC offload of L7 TrueTime pacing.

REFERENCES

[1] J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild, *et al.*, "Spanner: Google's globally distributed database," *ACM Transactions on Computer Systems (TOCS)*, vol. 31, no. 3, pp. 8:1–8:22, 2013.

[2] Anthropic, "Model context protocol specification," *Anthropic Technical Specification*, 2024. [Online]. Available: https://modelcontextprotocol.io

[3] Google, "Agent2Agent (A2A) protocol specification," *Google Cloud Architecture Center*, 2025. [Online]. Available: https://google.github.io/A2A/

[4] H. Garcia-Molina and K. Salem, "Sagas," in *Proc. ACM SIGMOD International Conference on Management of Data*, 1987, pp. 249–259.

[5] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[6] C. J. Fidge, "Timestamps in message-passing systems that preserve the partial ordering," in *Proc. 11th Australian Computer Science Conf. (ACSC)*, vol. 10, no. 1, 1988, pp. 56–66.

[7] F. Mattern, "Virtual time and global states of distributed systems," in *Proc. Int. Workshop Parallel and Distributed Algorithms*, 1988, pp. 215–226.

[8] S. S. Kulkarni, M. Demirbas, D. Madappa, B. Avva, and M. Leone, "Logical physical clocks and consistent snapshots in globally distributed databases," in *Proc. Int. Conf. Principles of Distributed Systems (OPODIS)*, 2014, pp. 17–32.

[9] M. P. Herlihy and J. M. Wing, "Linearizability: A correctness condition for concurrent objects," *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 12, no. 3, pp. 463–492, 1990.

[10] E. B. Nightingale, P. M. Chen, and J. Flinn, "Speculative execution in a distributed file system," in *Proc. 20th ACM Symp. Operating Systems Principles (SOSP)*, *ACM SIGOPS Operating Systems Review*, vol. 39, no. 5, 2005, pp. 191–205.

[11] A. Saeed, N. Dukkipati, V. Valancius, V. T. Lam, C. Contavalli, and A. Vahdat, "Carousel: Scalable traffic shaping at end hosts," in *Proc. ACM Special Interest Group on Data Communication (SIGCOMM)*, 2017, pp. 404–417.

[12] S. Radhakrishnan, Y. Geng, V. Jeyakumar, A. Kabbani, G. Porter, and A. Vahdat, "SENIC: Scalable NIC for end-host rate limiting," in *Proc. 11th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2014, pp. 475–488.

[13] S. G. Patil, T. Zhang, V. Fang, N. Chaurasia, R. Huang, A. Hao, M. Casado, J. E. Gonzalez, R. A. Popa, and I. Stoica, "GoEX: Perspectives and designs towards a runtime for autonomous LLM applications," *arXiv preprint arXiv:2404.06921*, 2024.

[14] E. Y. Chang and L. Geng, "SagaLLM: Context management, validation, and transaction guarantees for multi-agent LLM planning," *Proceedings of the VLDB Endowment (Proc. VLDB Endow.)*, vol. 18, 2025.

[15] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," *arXiv preprint arXiv:2406.12045*, 2024.

[16] N. Ye, A. Ahuja, G. Liargkovas, Y. Lu, K. Kaffes, and T. Peng, "Speculative Actions: A lossless framework for faster agentic systems," *arXiv preprint arXiv:2510.04371*, 2025.

[17] Y. Ro, H. Qiu, Z. Wang, E. Choukse, R. Fonseca, R. Bianchini, A. Akella, and M. Erez, "Sherlock: Reliable and efficient agentic workflow execution," *arXiv preprint arXiv:2511.00330*, 2025.

[18] B. Mohammadi, N. Potamitis, L. Klein, A. Arora, and L. Bindschaedler, "Atomix: Timely, transactional tool use for reliable agentic workflows," *arXiv preprint arXiv:2602.14849*, 2026.

[19] L. Geng and E. Y. Chang, "ALAS: Transactional and Dynamic Multi-Agent LLM Planning," *arXiv preprint arXiv:2511.03094*, 2025.